



AI Voice Agent for Inbound Carrier Call Handling

Delivered by: ASRP · **Role:** ASRP owned the voice-AI feature (design, implementation, roadmap) · **Engagement type:** Independent contractor · **Domain:** Cloud telephony for freight logistics

Anonymized case study. Client identity, proprietary source, internal identifiers, and security-sensitive details are omitted. Third-party platforms are named only where they are public products.

1. Executive summary

The client is a cloud telephony platform for freight logistics, sitting between **carriers** (trucking companies hauling freight) and **brokers** (who arrange it). On every inbound carrier call, the platform already enriches the call with business context: it looks the calling number up against its databases, checks whether it's tied to a known carrier, and runs a traditional IVR ("enter your MC number", "enter the load number") before the call reaches a human.

ASRP designed and built the **voice-AI agent** that upgrades that IVR. When a caller's load number can't be matched, the call hands off to a conversational agent that identifies the shipment by voice — reference number, or origin/destination and dates — and then reports a structured result so the backend can route a qualified call into the human queue. The agent is **conversational to the caller but strictly deterministic under the hood**: it never invents shipment data, it never decides the call's fate on its own, and it degrades gracefully back to the classic IVR if anything fails.

The system integrates the speech-native AI platform **Ultravox**¹ into the platform's existing **Jambonz**² + **Bandwidth**³ telephony stack over four independent communication channels, with real-time transcription, event-driven routing, and a dedicated testing / self-evolution harness.

2. Business context & the problem

The platform's value is turning a raw phone call into an **enriched, verified business event** — *who* is calling, from *which* carrier company, about *which* load, and *why*. Caller ID alone proves little, so the platform ties the number to MC/DOT identity, carrier authority status, associated carriers, prior calls, and compliance state before a human ever has to work it out.

The weak point was the **first stage of the call**. Baseline flow: the carrier calls a broker's number → enters an MC number and a load number in the IVR → gets routed to a human. By the time a human answers, they know **WHO** is calling (from the MC) and **WHAT** load (from the number entered) — but **not WHY**, and there is no continuity between calls. In practice that produced:

- **A missing "why" at handoff** — the human picks up without knowing the reason for the call.

- **No continuity** — “hey, transfer me back to the person I was just talking to” has no answer; callers phone back repeatedly to clarify or follow up.
- **Dead ends** — a rigid menu can’t handle a carrier who doesn’t have the exact load number, or a natural request like “I’m the driver for the Chicago pickup tomorrow.”

The objective: replace the rigid first stage with an agent that establishes the missing facts **before** a human is involved, and carries that context across the handoff.

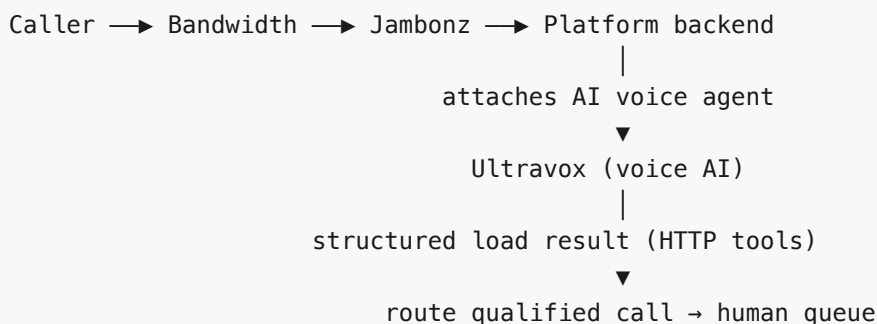
3. What ASRP was asked to do

The client already had a **call model** implemented in their backend (call state, legs, IVR). ASRP’s mandate was to extend it and make the front of the call intelligent:

1. **Extend the call model** so the backend can carry a call **classification** — **WHO / WHY / WHAT** — not just call plumbing.
2. **Make the IVR intelligent.** At the database level, associate the calling **number with a specific carrier** (so a future call can proactively ask “*are you calling on behalf of X?*”), track WHO/WHY/WHAT across calls, and on a **repeat call** immediately ask “*same carrier? same load?*” — giving the continuity the old flow lacked.
3. **Insert a switch-to-agent step** into the standard call flow: a voice agent the caller can tell *what* and *why* they’re calling, whose structured result lets the backend route the call to the right human.

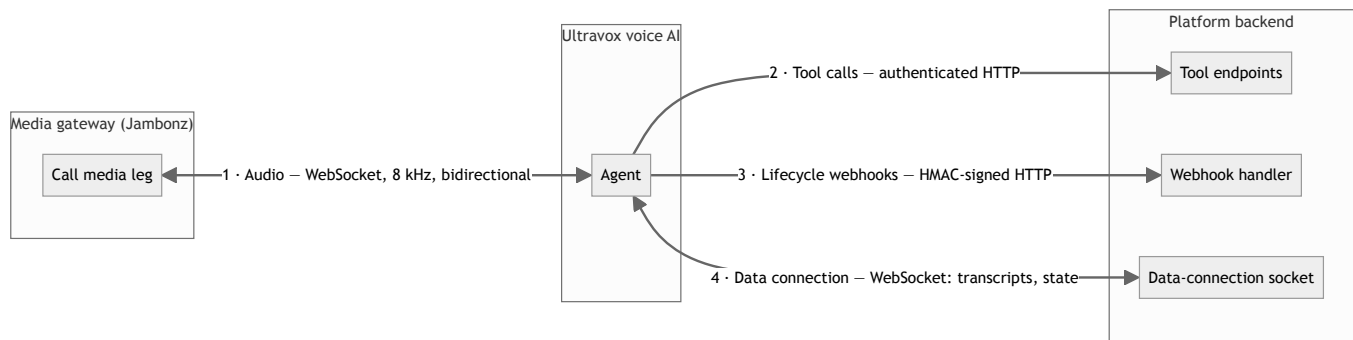
4. System architecture

The call spine is Caller → Bandwidth (telco/SIP) → Jambonz (media gateway) → platform backend . The backend attaches the AI agent to the live call and mediates every interaction with it.



The four-channel integration (core design)

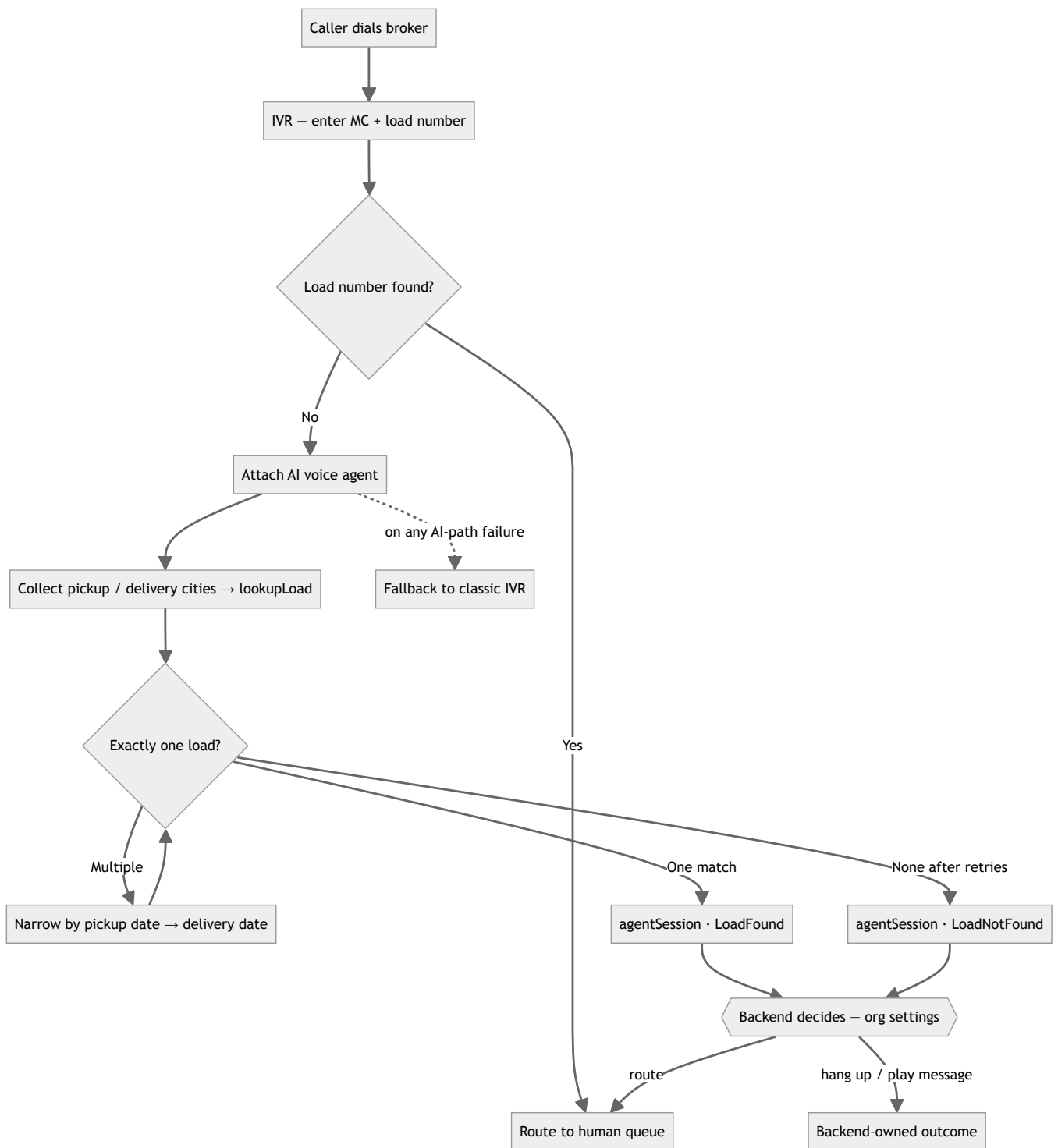
The essential design insight is that the AI and the backend communicate over **four independent channels**, each a different protocol with a different job. Cleanly separating them is what makes the integration robust.



#	Channel	Protocol	Purpose
1	Audio	WebSocket, 8 kHz, bidirectional	Voice media between gateway and AI
2	Tool calls	HTTP POST, Bearer-authenticated	Agent looks up loads/carriers and reports events; backend owns all data
3	Lifecycle webhooks	HTTP POST, HMAC-SHA256 signed	Backend tracks call state (<code>call.started</code> / <code>joined</code> / <code>ended</code>) independently of the conversation
4	Data connection	WebSocket	Real-time transcripts + control messages; transcripts persisted and broadcast live to the human-agent UI

5. The pilot agent

The pilot agent is deliberately **narrow and on-rails**. It is attached to a call **only when a caller has already provided a load number in the IVR and it was not found in the system** — so it joins mid-IVR, not as a general chatbot.



Its single job is to **identify exactly one shipment by voice**:

- Ask for pickup and delivery **cities/states**, then search.
- If multiple loads match, **narrow by pickup date, then delivery date**; if still ambiguous, read out the candidates and let the caller pick. Natural date expressions (“tomorrow”, “next Friday”) are normalized to YYYY-MM-DD .
- Product insight baked into the flow: find *a matching* load, not necessarily one exact record.

It runs on **two tools** — `lookupLoad` (search) and `agentSession` (event reporter) — and two guiding principles that make it trustworthy:

- **Report, don't decide.** The agent only *reports* what happened (`LoadFound` , `LoadNotFound` , and edge events); the **backend** decides whether to transfer or hang up, based on organization settings. The agent may not end the call.
- **Voice and tools are separate channels.** On a terminal result the agent calls the tool **silently and stops speaking** — the backend owns every caller-facing message. (A small touch: it says “*Found it, one second.*” purely to mask tool-call latency.)

Edge cases map to defined events rather than agent improvisation: `CallerCannotProvideInfo` , `Silence` , `CallerRequestedHangup` , `AbusiveCaller` , and `OffTopicQuestion` (rates/tracking/dispatch/payments are deflected once, then the flow continues).

6. What an actual call sounds like

(Every name, city, and reference number below is invented for this illustration — no real caller, carrier, or load data.)

The caller has already been through the IVR and entered a load number that didn't match — that's the trigger described in §5. Bandwidth and Jambonz hand the call to the agent mid-flow; the caller has no idea a different system just picked up the line.

Agent: “Sorry, I wasn't able to find a load with that number. Can you give me the pickup and delivery cities instead?”

Caller: “Uh, yeah — picking up in Modesto, dropping in Reno.”

(silently: `lookupLoad({ pickupCity: "Modesto", deliveryCity: "Reno" })` — two candidates come back)

Agent: “I've got two loads on that lane. Do you know the pickup date?”

Caller: “Tomorrow.”

(“Tomorrow” is anchored to the call's actual date and converted to `YYYY-MM-DD` before it ever reaches a tool call — never passed through as-is. Re-queried, narrowed to one match.)

Agent: “Found it, one second.”

(The agent starts the `agentSession(LoadFound)` tool call the instant it starts that sentence, not after — so the caller hears speech, not dead air, while the backend records the result and prepares the transfer. Then the agent goes silent: the backend speaks next, not the agent.)

Backend (queue transfer): “Connecting you now.”

A second excerpt shows the agent holding the rails when the caller drifts off-topic:

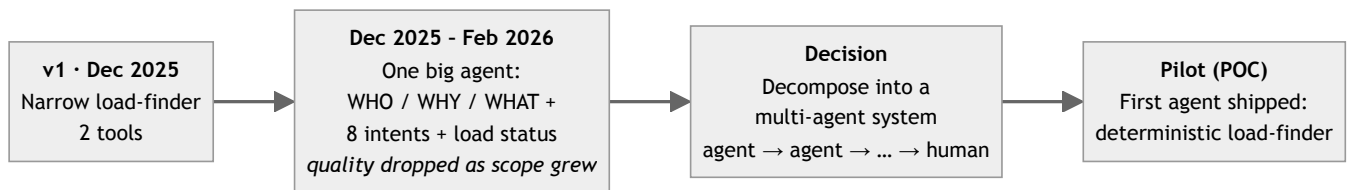
Caller: “Hey, while I've got you — what's the rate on this one?”

Agent: “I can't get into rates on this line, but let's get your load squared away first — what are the pickup and delivery cities?”

(`OffTopicQuestion` is logged and deflected once; the flow continues. The agent doesn't refuse the topic forever, and it doesn't wander into a conversation it has no authority over.)

7. From one big agent to a multi-agent system

The pilot's load-finder is one focused agent carved out of a much richer agent that ASRP built and exercised during the engagement. The evolution — and the key architectural decision behind it — went like this:



Along the way ASRP built the full first-stage capability set — initially inside one agent:

- **WHO — carrier identity.** MC number (primary), carrier name, caller name, and caller **type** (`driver` / `dispatcher` / `third_party_dispatch` / `owner_operator`). Carrier and caller are modeled **separately** (like *company* vs *contact* in a CRM), because one caller can represent multiple carriers. Known numbers are confirmed (“is this ABC Freight?”); unknown numbers are asked for the MC.
- **WHY — intent classification.** An 8-value intent taxonomy — `LOAD_INQUIRY` (the majority of calls), `CALLBACK`, `POST_BOOKING_SUPPORT`, `ONBOARDING_COMPLIANCE`, `IN_TRANSIT_STATUS`, `BILLING_PAYMENT`, `EXCEPTION_URGENT`, `OTHER_UNKNOWN` — each with a **confidence score**; below 0.70 the agent asks one clarifying question instead of guessing.
- **WHAT — load identification** with the multi-match narrowing funnel described above, plus **load-status handling**: `POSTED` (available → proceed), `UNPOSTED` (unavailable), `DELETED` (already covered) — the agent only completes on an available load.
- **Incremental events** that update backend call-state while the conversation continues, with a terminal completion event that triggers routing.

Why the pilot is a single load-finder. As that one agent’s responsibility grew — classify the intent, identify the carrier, find the load, handle every edge case — its quality on each individual task got *worse*. A bigger remit made a weaker agent. So rather than push one monolithic agent to do everything, the design decision was to **decompose it into a multi-agent system**: each agent owns a narrow job and can **hand the call off to another agent** when the task changes — agent to agent, until the call reaches a human. That decomposition is partially built (including a stage-based split explored in the framework); the product’s proof-of-concept **launched on the first agent in that system — the deterministic load-finder**. (Real-call analysis independently reinforced the case for narrow agents: a broad agent was brittle in a noisy phone channel — e.g. mis-transcribed MC numbers and conversational dates.)

Guardrails (why it’s trustworthy)

- The agent **may not end the call** — the backend controls termination.
- The agent **may not store load data** in its own memory.
- The agent **may not introduce facts** the caller didn’t state — every reported value is grounded in a backend lookup (a strict “use only what the caller said” data policy).
- Search is via the platform’s own REST/DB, **not** a vector store — deterministic, auditable results.

8. War stories — the hardest problems in development

Four things came up in real usage and testing that the architecture sections above don't show on their own.

1. MC numbers didn't survive the phone line. Spoken digit strings — MC numbers especially — are exactly the kind of input a phone-audio channel garbles: dropped digits, transposed digits, static. Real-call session analysis (captured by the testing framework in §13) surfaced this as a recurring failure mode, and it directly informed the decision in §7 to keep the shipped agent narrow rather than push a broad classifier at a noisy channel: a broad agent asked to get an MC number *and* an intent *and* a load right in one pass had nowhere to absorb that error. The mitigation baked into the prompt is a confidence threshold — below 0.70 the agent asks a clarifying question instead of guessing — rather than any attempt at audio-level correction.

2. Natural dates didn't parse the way callers said them. “Tomorrow,” “next Friday,” “day after tomorrow” — deceptively simple to a human, ambiguous to a model unless it's anchored. Early behavior sometimes passed the caller's words straight into a tool call, or resolved a relative date against the wrong reference point. The fix: the prompt carries an explicit `{{ todayDate }}` anchor plus a worked table of conversions (“tomorrow” → today + 1 day → YYYY-MM-DD) and paired correct/incorrect examples, so date normalization happens as a hard rule before any tool sees the value — never as an inference the agent makes on the fly.

3. The agent invented details the caller never said. The clearest hallucination risk wasn't fabricated load data — the guardrails in §7 already rule that out structurally — it was smaller: the agent would sometimes fill in a destination state the caller hadn't actually mentioned, inferring it from context or from a bare origin. The fix is a blunt data policy enforced in the prompt: use *only* the fields the caller explicitly said, field by field — “load from New Mexico” populates `originState` and nothing else, even though a destination might seem obvious.

4. Tool-call latency created dead air — solved by talking through it, not waiting through it. Every `lookupLoad / agentSession` call is a network round trip, and a silent multi-hundred-millisecond gap on a phone call reads as a hang or a dropped line. Instead of waiting for the tool result before speaking, the agent is prompted to speak a short bridging phrase (“Found it, one second.”) **while** firing the tool call in parallel — voice and tool call run concurrently, not sequentially. Get the order wrong — call the tool silently and only speak after — and it's called out as wrong behavior directly in the prompt's own worked examples: silence during a tool call reads as latency to a caller, even when the underlying call itself is fast.

9. Technology stack

Layer	Technology
Conversational voice AI	Ultravox (speech-native model, STT+LLM fused; model <code>ultravox-v0.7</code>)
Text-to-speech	InWorld , voice “Sarah” — matched to the platform’s Jambonz IVR voice for a seamless handoff
Programmable voice / media	Jambonz
Telco / SIP	Bandwidth
Backend	TypeScript , Node.js with the Hono web framework
Realtime transport	WebSockets (audio + data connection)
Event relay to UI	NATS pub/sub → WebSocket server → frontend
Persistence	MySQL via the Drizzle ORM — PlanetScale serverless in production, mysql2 (MySQL 9) for local dev; caching/queues on Redis
Testing harness	Standalone Bun/TypeScript project (Puppeteer for browser-driven full-voice tests)

On the AI model: Ultravox uses its own open-source, LLaMA-based model that understands voice tokens directly (speech-to-text and reasoning are fused, which cuts latency). The choice of underlying LLM is not user-selectable on the Ultravox platform — this was an accepted platform constraint, mitigated by strict prompt/tool determinism and backend-owned data.

10. Data model

The integration is backed by a relational SQL schema (Drizzle ORM). Three tables are central:

- **active_calls** — the central call-state row, keyed to the Ultravox session via an indexed `aiSessionId` correlation key. Holds org/routing context, call legs, carrier/caller/load fields populated by the agent’s tool flow (company name, load number, MC/DOT numbers), a human-readable `currentStatus`, and compliance/metadata columns. Collected data is also serialized as JSON on the call record.
- **ultravox_webhooks** — append-only log of lifecycle webhook events (`ultravoxCallId`, `webhookEvent`, full JSON payload), written **idempotently**, indexed by call id / event / time.
- **ultravox_transcripts** — the real-time transcript store: `role` (user/agent), `medium` (voice/text), `text`, `ordinal`, `final` flag, keyed by call id. Idempotency enforced on (`ultravoxCallId`, `ordinal`).

“Which database did you use?” → **MySQL**, accessed through the Drizzle ORM in TypeScript from a shared schema package. The platform runs a dual connection strategy: **PlanetScale serverless** (MySQL) in production and the **mysql2** driver (MySQL 9) for local development. ASRP added the tables above for the voice-AI feature and

extended the existing call-state table.

11. API & integration surface

Agent tools (AI → backend, authenticated HTTP POST):

Tool	Status	Purpose
lookupLoad	pilot	Load search by reference / cities / dates; returns candidate loads with status (posted/unposted/deleted)
agentSession	pilot	Reports classification & lifecycle events; the terminal event triggers backend routing
lookupEntity	built (broader config)	Carrier lookup by MC or DOT number; updates carrier fields on the call record

Lifecycle webhooks (AI → backend): `call.started`, `call.joined`, `call.ended`. The endpoint acknowledges immediately and processes in the background, writing each event idempotently. The `call.ended` handler reconciles the final state and ensures the call is queued even on fallback paths.

Security: webhooks are verified with **HMAC-SHA256** over `timestamp + "." + body`, with a signed-timestamp freshness window and **key-rotation support** (multiple valid signatures accepted during rotation). Tool calls are Bearer-token authenticated.

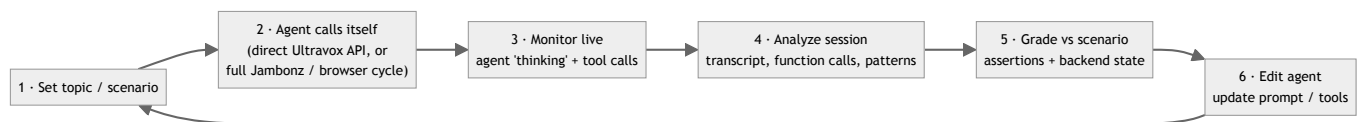
12. Reliability & operations

- **Feature-flagged end to end** — a single flag toggles the whole AI path; when off, the platform transparently uses the classic IVR.
- **Automatic fallback** — configurable per phone number; on any AI-path failure the call falls back to the rigid IVR automatically.
- **Idempotent writes** — webhook and transcript persistence are idempotent, so retried/duplicate deliveries never corrupt state.
- **Low-latency handoff** — a fast queue-placement path starts the human handoff the instant the load is identified, minimizing dead air.

13. The AI development loop (self-evolution framework)

Iterating on a voice agent by hand is slow: place a call, talk to it, guess what went wrong, edit the prompt, call again. ASRP built a standalone **Bun/TypeScript framework** that closes that loop and lets an AI in the IDE **develop the agent by itself** — a small meta-learning system where the agent is edited, called, and graded programmatically. It is a **separate repository** from the production platform (zero production dependency; it *extends* the shared Ultravox package with agent-update and prompt/tool utilities), so this tooling never ships into production code.

The loop is fully closed — the AI can edit the agent, call it, evaluate the call, and apply the fix, then repeat:



- **The agent calls and talks to itself.** A run sets a conversation topic or a predefined test case, then the agent places a call — either **direct to Ultravox** (bypassing the PBX; fast, for debugging behavior and tool calls) or through the **full Jambonz / browser pipeline** (creating a real call record + webhooks for end-to-end verification). A direct-WebSocket mode and a Puppeteer-driven full-voice mode round out the methods.
- **It watches the agent think.** Sessions are monitored in real time — chat history, tool/function calls, and tool-usage patterns — so failures are diagnosed from the agent’s actual reasoning, not guessed.
- **It grades against scenarios.** Named scenarios script the caller’s lines and carry explicit **assertions**: expected intent classification, extracted fields (load number, cities, MC), the tool calls that should fire, “the agent went silent after the terminal event,” and — for full-cycle runs — the resulting backend state (call queued, records written). Test cases span the intent taxonomy.
- **It edits the agent from the project.** Agent prompts and tools are updated programmatically from the framework, and an analysis pass can propose the changes to apply — so the diagnose → fix → re-test cycle runs without leaving the codebase.
- **Session corpus.** Every run is recorded (full transcript, events, and the complete system prompt in force) for offline analysis and regression review. This corpus is what surfaced the MC-digit and date-parsing issues covered in §8.

14. Design targets

The integration was engineered toward these voice-UX and accuracy goals (see §15 for pilot outcome):

Dimension	Target
Intent-classification latency	< 2 s
First agent response	< 3 s
Whole qualification phase	< 30 s
Tool-call latency	< 1 s
Intent-classification accuracy	> 90%
Caller/load capture rate	> 85%
Routing accuracy	> 95%
Uptime	> 99.5%

15. Results & outcome

The proof-of-concept worked. In live testing the agent handled real inbound calls end to end — conversing with callers, identifying loads by voice, and handing qualified calls to the human queue.

The system was delivered **production-ready but has not yet been switched on for live customer traffic**. Turning it on is a go-to-market decision on the client's side that had not happened by the end of the engagement — the build is complete and ready to enable whenever they choose to. The remaining step is a business decision, not further engineering.

16. Maturity & roadmap

Pilot scope (Phase 1 — deterministic): when a caller cannot provide a valid load number through the IVR, the call switches to the AI agent, which identifies the load by voice (reference number, or origin/destination + dates) and places the call in the human queue. Behavior is deliberately narrow and on-rails: minimal questions, no off-topic conversation.

Partially built — multi-agent decomposition: the target architecture is a set of focused agents that hand the call off agent-to-agent (until a human), rather than one monolithic agent whose quality degrades as its scope grows (see §7). The load-finder is the first agent; the handoff framework is partially built.

Designed but not yet shipped:

- **Backend routing decision engine** — ACCEPT / REJECT / HOLD based on compliance status, carrier-block state, load availability, org settings, and anti-spam (e.g. rate-limiting repeat callers). Rejection messages generated by the backend, not the agent.
- **ANI intelligence** — a caller-ID lookup cache and dedicated intent-classification tables, so a known number is recognized instantly and prior calls inform the conversation.

Future phases:

- **Phase 2 (conversational):** negotiation, bids, status updates once WHO/WHY/WHAT are known.
- **Phase 3 (outbound):** the agent initiates calls.

17. What ASRP delivered

ASRP owned the **entire voice-AI workstream** on top of the client's existing call platform:

- Authored the **strategic roadmap** for the two-phase (deterministic → conversational) voice architecture and the WHO/WHY/WHAT model.
- Designed and implemented all four AI↔backend communication channels (audio attach, HTTP tools, signed webhooks, live data connection).
- Extended the backend call model to carry call classification, and built the agent tool interface (`lookupLoad` , `agentSession` , `lookupEntity`).
- Built and iterated the agent behavior — from the full 8-intent classifier down to the deterministic pilot load-finder — authoring the system prompt and tool configuration against recorded sessions.
- Implemented real-time transcript capture with idempotent persistence and live broadcast to the human-agent UI.

- Built the standalone testing / self-evolution framework.

18. FAQ (for client conversations)

Q: What's actually running in the pilot? A narrow, deterministic agent: it takes over only when a caller's load number wasn't found, identifies the shipment by voice, and reports the result so the backend can route the call. The fuller intent classifier was built and iterated but held behind this reliable subset.

Q: What database did you use? MySQL via the Drizzle ORM in TypeScript — PlanetScale serverless in production, the mysql2 driver (MySQL 9) locally. Dedicated tables for call state, webhook logging, and transcripts, with Redis for caching/queues.

Q: Which LLM powers the agent? Ultravox's own speech-native model (LLaMA-based, voice-token-aware). The platform doesn't expose LLM choice; we compensated with strict prompt determinism, a fixed tool interface, and backend-owned data so the model converses but never fabricates.

Q: How do you stop it from hallucinating shipment data? Hard guardrails: the agent can only report values returned by backend lookups, cannot invent details, and cannot store data in its own memory. Search is deterministic (backend REST/DB, not a vector store).

Q: What happens if the AI fails mid-call? Automatic per-number fallback to the classic IVR. The whole feature is behind a single flag, so it can be disabled instantly without a deploy.

Q: Is it secure? Webhooks are HMAC-SHA256 signed with a freshness window and key rotation; tool calls are Bearer-authenticated; all event/transcript writes are idempotent.

Q: How is it tested? A dedicated framework with four test methods (from direct-socket to full browser-driven voice), scripted scenarios with assertions, and a recorded session corpus for regression analysis.

Anonymized project description prepared by ASRP for portfolio use. Client name, proprietary source paths, internal identifiers, and production infrastructure details are intentionally omitted from the body. Deeper technical detail is available under NDA.

1. **Ultravox** — a speech-native voice-AI platform whose model consumes audio directly (speech recognition and reasoning are fused rather than piped through a separate transcription step), which lowers conversational latency. <https://www.ultravox.ai>↩
2. **Jambonz** — an open-source programmable-voice / CPaaS platform that bridges telephony (SIP/PSTN carriers) to applications and voice bots, acting as the media gateway between the phone network and the AI. <https://jambonz.org>↩
3. **Bandwidth** — a US communications carrier / CPaaS providing SIP trunking and PSTN connectivity (phone numbers and voice) — the telco layer that carries the actual phone call. <https://www.bandwidth.com>↩